Introduction to Spring Framework

*Note: Look up the meanings of the terms in **bold**.*

Introduction

What is a framework?

A framework enables a coding environment that contains low-level libraries to address conventional coding issues. The objective of a framework is to deliver faster development of an application. This includes everything we need to build large-scale applications, such as templates based on best practices.

What is Spring?

Spring is a comprehensive framework for building Java applications. Spring is well known for being very configurable and less **opinionated** thereby allowing the users of the framework complete control over how the framework should behave according to the users' needs.

Although this flexibility has its advantages, when a need arises to create an application quickly, the same flexibility can become a hinderance which is the problem solved by Spring Boot.

What is Spring Boot?

As mentioned before, too much choice can also be a hindrance if you want just everything to work out of the box. That is exactly the need Spring Boot caters to. Spring Boot is an **opinionated** extension of the Spring framework with working configurations out of the box along with third-party library **integrations** allowing developers to focus on implementing actual business logic.

Spring Boot makes it easy to create stand-alone, production-grade Spring based Applications that you can "just run".

Prerequisites

Please have the following software set up in your computer before continuing.

Essential Software

- Java Development Kit (JDK) 18 or later
- Visual Studio Code (VS Code)

After VS Code is set up, please install the following extensions to VS Code as well.

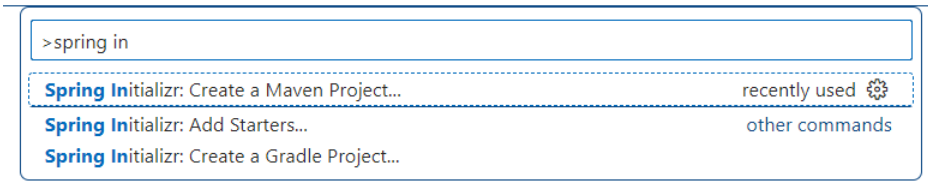
- Spring Initializr v0.11.2 (by Microsoft)
- Extension Pack for Java v0.25.7 (by Microsoft)

Creating a simple Spring Boot Application

Part 1 – Create a new Spring Boot Project

1. Open the command palette in VS Code (Ctrl + Shift + P in Windows).

2. Type "Spring initializr" and select *Spring Initializr: Create a Maven Project*



3. Select the latest Spring Boot version. Make sure not to select versions labelled as *SNAPSHOT*.

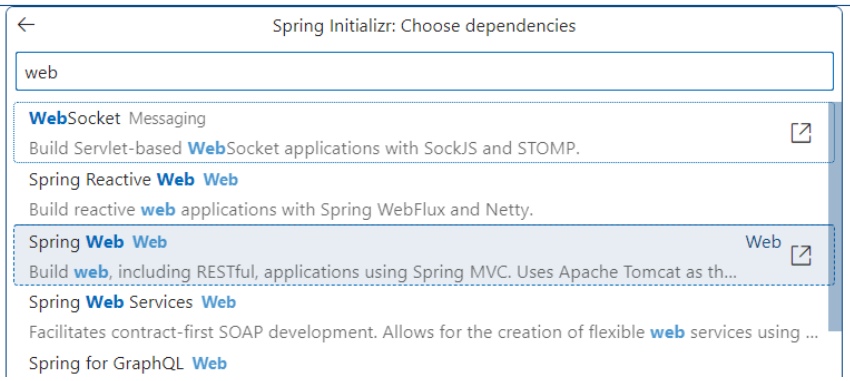
4. Select *Java* as the Project language.

5. Give a name for the *group id and artifact id*. See here for an explanation on these.

6. Select *Jar* as the packaging type for your project.

7. Select the *Java Version* (select one which has been installed on your computer)

8. Select *Dependencies* for the project. For this exercise, select only the *Spring Web* dependency.



9. Then press enter and select the folder path for creating the Spring Boot project.

Part 2 – Creating a Hello World! Application

1. Open the relevant VS Code project and locate the *<artifactid>Application.java* available under *src/main/java/com/<groupid>/<artifactid>* and open it.

2. Add the method *rootEndpoint()* and the ***annotations*** (highlighted) in this file just after the main method (see the image below).

```java
@SpringBootApplication
@RestController
public class DemoApplication {

    public static void main(String[] args) {
        SpringApplication.run(DemoApplication.class, args);
    }

    @GetMapping("/")
    public String rootEndpoint() {
        String message = "Hello world!";
        return message;
    }
```

The *@RestController* annotation tells Spring that this code describes an **endpoint** that should be made available over the web. The *@GetMapping("/")* tells Spring to use our *rootEndpoint()* method to answer requests that get sent to the *http://localhost:8080/* address.

3. Some errors will be thrown regarding missing packages etc. Make sure those are fixed using the QuickFix option in VS Code.

4. Once all errors have been fixed, run the project on VS Code. It should execute successfully.

5. Open your browser and navigate to *http://localhost:8080*. You will see "Hello world!" displayed on the page.

   Note: In case you run into an error due to port numbers, open *application.properties* file and add this line: *server.port=8081.* Save and re-run the project.

Part 3 – Self Guided Activity

1. Modify the above file to respond to an endpoint *"/hello"*.
   E.g., *http://localhost:8080/hello*.

2. Then, modify the method to display a name passed on as a **Query Parameter** along with Hello.
   E.g., *http://localhost:8080/hello?name=Amith*

Hint: See this link for help.